Arcturus Pipeline Integration Template

Assembles a four-layer production pipeline (feature ingestion, CI/CD deployment, drift monitoring, telemetry cockpit) and runs it under a simulated stress test.

All data is synthetic. CI/CD registration and the Databricks Jobs REST API trigger are simulated with stub functions in this lab environment, since no live Azure ML workspace or Databricks Jobs token is provisioned here -- see the comments on each stub for what the real Azure ML SDK v2 / REST API call looks like.

Execution environment for this submission: local Jupyter kernel (Python 3) with a local MLflow tracking store. The two Databricks-only calls (`dbutils.library.restartPython()` and the `dbutils` username lookup) are wrapped so the same notebook still runs unchanged on Azure Databricks; locally the kernel starts fresh, so no restart is needed. The stress test harness cell is unmodified.

Before you begin: environment setup (Azure Databricks)

In [1]:

```
%pip install fakeredis mlflow scipy pandas pyarrow
```

```
Requirement already satisfied: fakeredis in /usr/local/lib/python3.13/dist-packages (2.40.0)
Requirement already satisfied: mlflow in /usr/local/lib/python3.13/dist-packages (3.17.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.13/dist-packages (1.18.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (3.0.5)
Requirement already satisfied: pyarrow in /usr/local/lib/python3.13/dist-packages (25.0.1)
Requirement already satisfied: redis>=4.3 in /usr/local/lib/python3.13/dist-packages (from fa
keredis) (8.1.0)
Requirement already satisfied: sortedcontainers>=2 in /usr/local/lib/python3.13/dist-packages
(from fakeredis) (2.4.0)

Requirement already satisfied: mlflow-skinny==3.17.1 in /usr/local/lib/python3.13/dist-packag
es (from mlflow) (3.17.1)
Requirement already satisfied: mlflow-tracing==3.17.1 in /usr/local/lib/python3.13/dist-packa
ges (from mlflow) (3.17.1)
Requirement already satisfied: Flask-CORS<7 in /usr/local/lib/python3.13/dist-packages (from
mlflow) (6.0.5)
Requirement already satisfied: Flask<4 in /usr/local/lib/python3.13/dist-packages (from mlflo
w) (3.1.3)
Requirement already satisfied: aiohttp<4,>=3.7.0 in /usr/local/lib/python3.13/dist-packages (
from mlflow) (3.14.5)
Requirement already satisfied: alembic!=1.10.0,<2 in /usr/local/lib/python3.13/dist-packages
(from mlflow) (1.20.0)
Requirement already satisfied: cryptography<51,>=43.0.0 in /usr/local/lib/python3.13/dist-pac
kages (from mlflow) (50.0.1)
Requirement already satisfied: docker<8,>=4.0.0 in /usr/local/lib/python3.13/dist-packages (f
rom mlflow) (7.2.0)
Requirement already satisfied: graphene<4 in /usr/local/lib/python3.13/dist-packages (from ml
flow) (3.4.3)
Requirement already satisfied: gunicorn<27 in /usr/local/lib/python3.13/dist-packages (from m
lflow) (26.2.0)
Requirement already satisfied: huey<4,>=2.5.4 in /usr/local/lib/python3.13/dist-packages (fro
m mlflow) (3.4.0)
Requirement already satisfied: matplotlib<4 in /usr/local/lib/python3.13/dist-packages (from
mlflow) (3.11.2)
Requirement already satisfied: numpy<3 in /usr/local/lib/python3.13/dist-packages (from mlflo
w) (2.5.3)
```

```
Requirement already satisfied: scikit-learn<2 in /usr/local/lib/python3.13/dist-packages (fro
m mlflow) (1.9.1)
Requirement already satisfied: skops<1 in /usr/local/lib/python3.13/dist-packages (from mlflo
w) (0.16.0)
Requirement already satisfied: sqlalchemy<3,>=1.4.0 in /usr/local/lib/python3.13/dist-package
s (from mlflow) (2.1.5)
Requirement already satisfied: anyio!=4.15.0,<5,>=3.6.2 in /usr/local/lib/python3.13/dist-pac
kages (from mlflow-skinny==3.17.1->mlflow) (4.15.1)
Requirement already satisfied: cachetools<8,>=5.0.0 in /usr/local/lib/python3.13/dist-package
s (from mlflow-skinny==3.17.1->mlflow) (7.2.2)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.13/dist-packages (from
mlflow-skinny==3.17.1->mlflow) (8.5.0)
Requirement already satisfied: cloudpickle<4 in /usr/local/lib/python3.13/dist-packages (from
mlflow-skinny==3.17.1->mlflow) (3.1.2)
Requirement already satisfied: databricks-sdk<1,>=0.20.0 in /usr/local/lib/python3.13/dist-pa
ckages (from mlflow-skinny==3.17.1->mlflow) (0.153.0)
Requirement already satisfied: fastapi<1 in /usr/local/lib/python3.13/dist-packages (from mlf
low-skinny==3.17.1->mlflow) (0.143.3)
Requirement already satisfied: gitpython<4,>=3.1.9 in /usr/local/lib/python3.13/dist-packages
(from mlflow-skinny==3.17.1->mlflow) (3.2.1)

Requirement already satisfied: importlib_metadata!=4.7.0,<10,>=3.7.0 in /usr/local/lib/python
3.13/dist-packages (from mlflow-skinny==3.17.1->mlflow) (9.0.1)
Requirement already satisfied: opentelemetry-api<3,>=1.9.0 in /usr/local/lib/python3.13/dist-
packages (from mlflow-skinny==3.17.1->mlflow) (1.45.1)
Requirement already satisfied: opentelemetry-proto<3,>=1.9.0 in /usr/local/lib/python3.13/dis
t-packages (from mlflow-skinny==3.17.1->mlflow) (1.45.1)
Requirement already satisfied: opentelemetry-sdk<3,>=1.9.0 in /usr/local/lib/python3.13/dist-
packages (from mlflow-skinny==3.17.1->mlflow) (1.45.1)
Requirement already satisfied: packaging<27 in /usr/local/lib/python3.13/dist-packages (from
mlflow-skinny==3.17.1->mlflow) (26.3)
Requirement already satisfied: protobuf<8,>=3.12.0 in /usr/local/lib/python3.13/dist-packages
(from mlflow-skinny==3.17.1->mlflow) (7.36.1)
Requirement already satisfied: pydantic<3,>=2.0.0 in /usr/local/lib/python3.13/dist-packages
(from mlflow-skinny==3.17.1->mlflow) (2.13.5)
Requirement already satisfied: python-dotenv<2,>=0.19.0 in /usr/local/lib/python3.13/dist-pac
kages (from mlflow-skinny==3.17.1->mlflow) (1.2.3)
Requirement already satisfied: pyyaml<7,>=5.1 in /usr/local/lib/python3.13/dist-packages (fro
m mlflow-skinny==3.17.1->mlflow) (6.0.3)
Requirement already satisfied: requests<3,>=2.17.3 in /root/.local/lib/python3.13/site-packag
es (from mlflow-skinny==3.17.1->mlflow) (2.34.2)
Requirement already satisfied: sqlparse<1,>=0.4.0 in /usr/local/lib/python3.13/dist-packages
(from mlflow-skinny==3.17.1->mlflow) (0.6.0)
Requirement already satisfied: starlette<2 in /usr/local/lib/python3.13/dist-packages (from m
lflow-skinny==3.17.1->mlflow) (1.6.0)
Requirement already satisfied: typing-extensions<5,>=4.0.0 in /usr/local/lib/python3.13/dist-
packages (from mlflow-skinny==3.17.1->mlflow) (4.16.0)
Requirement already satisfied: uvicorn<1 in /usr/local/lib/python3.13/dist-packages (from mlf
low-skinny==3.17.1->mlflow) (0.53.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /root/.local/lib/python3.13/site-pac
kages (from pandas) (2.9.0.post0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.13/dist-pack
ages (from aiohttp<4,>=3.7.0->mlflow) (2.7.1)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.13/dist-packages (f
rom aiohttp<4,>=3.7.0->mlflow) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.13/dist-packages (from
aiohttp<4,>=3.7.0->mlflow) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.13/dist-packages (
from aiohttp<4,>=3.7.0->mlflow) (1.8.0)
Requirement already satisfied: multidict<8.0,>=4.5 in /usr/local/lib/python3.13/dist-packages
(from aiohttp<4,>=3.7.0->mlflow) (7.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.13/dist-packages (f
rom aiohttp<4,>=3.7.0->mlflow) (0.5.4)
Requirement already satisfied: yarl<2.0,>=1.25.1 in /usr/local/lib/python3.13/dist-packages (
from aiohttp<4,>=3.7.0->mlflow) (1.25.1)
Requirement already satisfied: Mako in /usr/local/lib/python3.13/dist-packages (from alembic!
=1.10.0,<2->mlflow) (1.4.3)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from c
ryptography<51,>=43.0.0->mlflow) (2.1.1)
```

```
Requirement already satisfied: urllib3>=1.26.0 in /root/.local/lib/python3.13/site-packages (
from docker<8,>=4.0.0->mlflow) (2.8.0)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.13/dist-packages (fro
m Flask<4->mlflow) (1.9.0)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.13/dist-packages
(from Flask<4->mlflow) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /root/.local/lib/python3.13/site-packages (fr
om Flask<4->mlflow) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /root/.local/lib/python3.13/site-packages
(from Flask<4->mlflow) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.13/dist-packages (fr
om Flask<4->mlflow) (3.1.8)
Requirement already satisfied: graphql-core<3.3,>=3.1 in /usr/local/lib/python3.13/dist-packa
ges (from graphene<4->mlflow) (3.2.13)
Requirement already satisfied: graphql-relay<3.3,>=3.1 in /usr/local/lib/python3.13/dist-pack
ages (from graphene<4->mlflow) (3.2.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (f
rom matplotlib<4->mlflow) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from
matplotlib<4->mlflow) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /usr/local/lib/python3.13/dist-packages (
from matplotlib<4->mlflow) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (
from matplotlib<4->mlflow) (1.5.1)
Requirement already satisfied: pillow>=9 in /usr/local/lib/python3.13/dist-packages (from mat
plotlib<4->mlflow) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /usr/local/lib/python3.13/dist-packages (from
matplotlib<4->mlflow) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from pyth
on-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: joblib>=1.4.0 in /usr/local/lib/python3.13/dist-packages (from
scikit-learn<2->mlflow) (1.6.0)

Requirement already satisfied: narwhals>=2.0.1 in /usr/local/lib/python3.13/dist-packages (fr
om scikit-learn<2->mlflow) (2.26.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in /usr/local/lib/python3.13/dist-package
s (from scikit-learn<2->mlflow) (3.7.0)
Requirement already satisfied: prettytable>=3.9 in /usr/local/lib/python3.13/dist-packages (f
rom skops<1->mlflow) (3.18.0)

Requirement already satisfied: idna>=2.8 in /root/.local/lib/python3.13/site-packages (from a
nyio!=4.15.0,<5,>=3.6.2->mlflow-skinny==3.17.1->mlflow) (3.20)
Requirement already satisfied: pycparser in /usr/local/lib/python3.13/dist-packages (from cff
i>=2.0.0->cryptography<51,>=43.0.0->mlflow) (3.0)

Requirement already satisfied: google-auth~=2.0 in /usr/local/lib/python3.13/dist-packages (f
rom databricks-sdk<1,>=0.20.0->mlflow-skinny==3.17.1->mlflow) (2.62.0)

Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-pac
kages (from fastapi<1->mlflow-skinny==3.17.1->mlflow) (0.4.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-package
s (from fastapi<1->mlflow-skinny==3.17.1->mlflow) (0.0.5)

Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.13/dist-packages (fr
om gitpython<4,>=3.1.9->mlflow-skinny==3.17.1->mlflow) (4.0.12)
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.13/dist-packages (from im
portlib_metadata!=4.7.0,<10,>=3.7.0->mlflow-skinny==3.17.1->mlflow) (4.1.1)

Requirement already satisfied: opentelemetry-semantic-conventions==0.66b1 in /usr/local/lib/p
ython3.13/dist-packages (from opentelemetry-sdk<3,>=1.9.0->mlflow-skinny==3.17.1->mlflow) (0.
66b1)
Requirement already satisfied: wcwidth>=0.3.5 in /usr/local/lib/python3.13/dist-packages (fro
m prettytable>=3.9->skops<1->mlflow) (0.9.2)

Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packa
ges (from pydantic<3,>=2.0.0->mlflow-skinny==3.17.1->mlflow) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packag
es (from pydantic<3,>=2.0.0->mlflow-skinny==3.17.1->mlflow) (2.46.5)
Requirement already satisfied: charset_normalizer<4,>=2 in /root/.local/lib/python3.13/site-p
ackages (from requests<3,>=2.17.3->mlflow-skinny==3.17.1->mlflow) (3.5.1)
Requirement already satisfied: certifi>=2023.5.7 in /root/.local/lib/python3.13/site-packages
(from requests<3,>=2.17.3->mlflow-skinny==3.17.1->mlflow) (2026.7.22)

Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.13/dist-packages (from uvic
orn<1->mlflow-skinny==3.17.1->mlflow) (0.16.0)

Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.13/dist-packages (fr
om gitdb<5,>=4.0.1->gitpython<4,>=3.1.9->mlflow-skinny==3.17.1->mlflow) (5.0.3)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.13/dist-packag
es (from google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.17.1->mlflow) (0.4.2)
```

```
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.13/dist-package
s (from pyasn1-modules>=0.2.1->google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.
17.1->mlflow) (0.6.4)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting beha
viour with the system package manager. It is recommended to use a virtual environment instead
: https://pip.pypa.io/warnings/venv

Note: you may need to restart the kernel to use updated packages.
```

`numpy` is preinstalled on Databricks ML Runtime clusters.

In [2]:

```python
try:
    dbutils.library.restartPython()  # Azure Databricks: restart so the %pip installs load
except NameError:
    print("Not running on Databricks: this kernel was started fresh, so no restart is needed.")
```

```
Not running on Databricks: this kernel was started fresh, so no restart is needed.
```

This restart clears your Python session. Run every cell below from the top, in order, without skipping any -- do not re-run this restart cell again once you continue past it, or you will need to re-run everything below it again.

Step 1: Imports, configuration, and baseline feature snapshot

Upload `arcturus_synthetic_load_config.json` to this notebook's folder before running the cell below. `feature_baseline_snapshot.parquet` (the Delta table stand-in described in `arcturus_delta_schema_reference.md`) is generated here so the rest of the notebook has real data to compare against -- on a real Databricks workspace with Delta Lake provisioned, replace this with `spark.read.format("delta").load(...)` as shown in the schema reference.

In [3]:

```python
import json
import time
import random
import uuid

import numpy as np
import pandas as pd
import fakeredis
import mlflow

with open("arcturus_synthetic_load_config.json") as f:
    config = json.load(f)

FEATURE_COLUMNS = config["feature_columns"]
SHIFTED_COLUMN = config["shifted_feature_column"]
PSI_THRESHOLD = config["psi_threshold"]
PSI_CHECK_INTERVAL = config["psi_check_interval_requests"]
PSI_WINDOW_SIZE = config["psi_rolling_window_size"]
TOTAL_REQUESTS = config["total_simulated_requests"]
SHIFT_START = config["covariate_shift_start_request"]
SHIFT_ROWS = config["covariate_shift_row_count"]
CACHE_MISS_RATE = config["redis_cache_miss_rate"]
P95_SLA_MS = config["p95_latency_sla_ms"]

np.random.seed(42)
N_BASELINE = 20000
baseline_data = {"item_id": [f"item_{i}" for i in range(N_BASELINE)]}
for col in FEATURE_COLUMNS:
    baseline_data[col] = np.random.beta(2, 5, N_BASELINE)
baseline_df = pd.DataFrame(baseline_data)
baseline_df.to_parquet("feature_baseline_snapshot.parquet")

N_ITEMS = 5000
sample_items = baseline_df.sample(N_ITEMS, random_state=1).reset_index(drop=True)
print(f"Baseline snapshot: {len(baseline_df)} rows. Sampled {N_ITEMS} items for the serving corpus.")
```

```
Baseline snapshot: 20000 rows. Sampled 5000 items for the serving corpus.
```

Layer 1: Feature Ingestion (Integration Point 1)

Seed a local Redis-compatible emulator with the sampled items, and set up the Delta feature store fallback for cache misses.

In [4]:

```
redis_client = fakeredis.FakeStrictRedis()
for _, row in sample_items.iterrows():
    values = ",".join(str(row[c]) for c in FEATURE_COLUMNS)
    redis_client.set(f"item_features:{row['item_id']}", values)

delta_feature_store = sample_items.set_index("item_id")

# INTEGRATION POINT 1 (completed): feature ingestion -> serving layer handshake
random.seed(42)  # makes the simulated cache-miss draws reproducible run to run

def hydrate_feature_vector(item_id):
    """Return (feature_vector, was_cache_miss) for one item.

    Hot path: Azure Cache for Redis (fakeredis stand-in). Fallback: the Delta feature
    store. A simulated miss, a missing key, or a Redis error all degrade to the
    feature store instead of breaking the serving loop.
    """
    if random.random() >= CACHE_MISS_RATE:  # simulated cache hit
        try:
            raw = redis_client.get(f"item_features:{item_id}")
        except Exception:  # connection/timeout errors fall through to the feature store
            raw = None
        if raw is not None:
            return [float(v) for v in raw.decode("utf-8").split(",")], False
    # Cache miss: read the same six columns from the Delta feature store
    row = delta_feature_store.loc[item_id, FEATURE_COLUMNS]
    return [float(v) for v in row.to_numpy()], True
```

In [5]:

```
# Ingestion self-check: both paths return the same mutable 6-float list for an item,
# and a key missing from Redis degrades to the feature store instead of raising.
test_item = sample_items.loc[0, "item_id"]
expected = [float(v) for v in delta_feature_store.loc[test_item, FEATURE_COLUMNS].to_numpy()]

paths = {"hit": 0, "miss": 0}
for _ in range(2000):
    vec, was_miss = hydrate_feature_vector(test_item)
    assert isinstance(vec, list) and len(vec) == len(FEATURE_COLUMNS)
    assert vec == expected
    paths["miss" if was_miss else "hit"] += 1

key = f"item_features:{test_item}"
saved = redis_client.get(key)
redis_client.delete(key)
vec, was_miss = hydrate_feature_vector(test_item)
assert was_miss and vec == expected
redis_client.set(key, saved)

print(f"2,000 lookups -> hits: {paths['hit']}, misses: {paths['miss']} "
      f"(configured miss rate {CACHE_MISS_RATE:.0%}); both paths return identical vectors.")
print("Missing Redis key -> served from the Delta feature store, no exception raised.")
```

```
2,000 lookups -> hits: 1915, misses: 85 (configured miss rate 5%); both paths return identica
l vectors.
Missing Redis key -> served from the Delta feature store, no exception raised.
```

Layer 2: CI/CD Deployment (Integration Point 3)

These are stub functions standing in for real Azure ML SDK v2 calls, since this lab environment has no live Azure ML workspace to authenticate against. Each stub's docstring shows the real SDK v2 call it replaces.

In [6]:

```
def register_model_stub(mlflow_run_id):
    """Stand-in for:
    ml_client.models.create_or_update(Model(name=..., path=..., tags={'mlflow_run_id': mlflow_run_id}))
    """
    model_version = str(uuid.uuid4())[:8]
    return {"model_name": "arcturus-ranking-model", "version": model_version, "tagged_run_id": mlflow_run_
id}

def update_endpoint_stub(model_version):
    """Stand-in for:
    ml_client.online_deployments.begin_create_or_update(
        ManagedOnlineDeployment(name=..., endpoint_name='arcturus-prod-endpoint', model=model_version)
    )
    """
    return {"endpoint": "arcturus-prod-endpoint", "deployed_version": model_version, "status": "Updated"}
```

Layer 3: Statistical Drift Monitoring (Integration Point 2)

PSI is computed periodically on a rolling window of recently served feature values, not per-request -- this keeps the drift loop from adding latency to the serving path measured for the p95 SLA.

In [7]:

```
# INTEGRATION POINT 2 (completed): drift monitoring -> retraining trigger
# The drift loop reads its reference distribution from the Delta table stand-in described
# in arcturus_delta_schema_reference.md (feature_baseline_snapshot). On a Databricks
# workspace with Delta Lake provisioned this line becomes:
#   spark.read.format("delta").load("/mnt/arcturus/feature_baseline_snapshot").toPandas()
baseline_df = pd.read_parquet("feature_baseline_snapshot.parquet")

PSI_EPSILON = 1e-4  # floor for empty-bin percentages so ln() stays finite

def calculate_psi(baseline_values, live_values, n_bins=10):
    """Population Stability Index over n_bins equal-width buckets spanning [0, 1].

    PSI = sum((live_pct - baseline_pct) * ln(live_pct / baseline_pct))
    """
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    baseline_counts, _ = np.histogram(np.clip(baseline_values, 0.0, 1.0), bins=edges)
    live_counts, _ = np.histogram(np.clip(live_values, 0.0, 1.0), bins=edges)
    baseline_pct = np.clip(baseline_counts / len(baseline_values), PSI_EPSILON, None)
    live_pct = np.clip(live_counts / len(live_values), PSI_EPSILON, None)
    return float(np.sum((live_pct - baseline_pct) * np.log(live_pct / baseline_pct)))

def trigger_retraining_job_stub():
    """Stand-in for a POST to the Azure Databricks Jobs REST API
    (POST /api/2.1/jobs/run-now with a bearer token) -- no live Databricks
    workspace token is available in this lab environment.
    """
    return str(uuid.uuid4())[:8] + "-prod-retrain"
```

In [8]:

```
# PSI self-check. Uses its own generator so the global NumPy stream used by the
# stress test harness is left untouched.
check_rng = np.random.default_rng(7)
base = baseline_df[SHIFTED_COLUMN].values
same_dist = check_rng.choice(base, PSI_WINDOW_SIZE, replace=False)
shifted = check_rng.beta(6, 1.5, PSI_WINDOW_SIZE)
print(f"Baseline rows read from feature_baseline_snapshot.parquet: {len(baseline_df)}")
print(f"PSI, {PSI_WINDOW_SIZE}-row window from the baseline itself : {calculate_psi(base, same_dist):.4f}"
)
print(f"PSI, {PSI_WINDOW_SIZE}-row window from the shifted distribution: {calculate_psi(base, shifted):.4f
}")
print(f"Retraining threshold (psi_threshold): {PSI_THRESHOLD}")
```

```
Baseline rows read from feature_baseline_snapshot.parquet: 20000
PSI, 1000-row window from the baseline itself : 0.0019
PSI, 1000-row window from the shifted distribution: 8.9326
Retraining threshold (psi_threshold): 0.25
```

Layer 4: Telemetry Cockpit (Integration Point 4)

In [9]:

```
def simulate_engagement_metrics(drift_active):
    penalty = 1.4 if drift_active else 1.0
    saves_rate = max(0.0, np.random.normal(0.12, 0.02) / penalty)
    dwell_time_median_s = max(5.0, np.random.normal(48, 5) / penalty)
    private_shares_rate = max(0.0, np.random.normal(0.03, 0.01) / penalty)
    return saves_rate, dwell_time_median_s, private_shares_rate
```

Step 2: MLflow experiment setup

In [10]:

```
try:
    # Azure Databricks: workspace experiment under the signed-in user
    username = dbutils.notebook.entry_point.getDbutils().notebook().getContext().userName().get()
    EXPERIMENT_NAME = f"/Users/{username}/arcturus-pipeline-validation"
except NameError:
    # Local kernel: same experiment name in the local MLflow tracking store
    EXPERIMENT_NAME = "arcturus-pipeline-validation"

experiment = mlflow.set_experiment(EXPERIMENT_NAME)
print(f"MLflow experiment: {experiment.name} (id {experiment.experiment_id})")
```

```
2026/10/17 02:28:05 INFO mlflow.store.db.utils: Creating initial MLflow database tables...

2026/10/17 02:28:05 INFO mlflow.store.db.utils: Updating database tables

2026/10/17 02:28:06 INFO mlflow.tracking.fluent: Experiment with name 'arcturus-pipeline-vali
dation' does not exist. Creating a new experiment.

MLflow experiment: arcturus-pipeline-validation (id 1)
```

Step 3-4: Stress Test Harness

Do not modify this cell. Simulates `total_simulated_requests` (corresponding to `qps_target x stress_test_duration_sec`) processed as fast as possible, with a covariate shift injected into `shifted_feature_column` for a bounded window of requests. The drift check runs periodically, not per-request, and the retraining trigger fires at most once per run.

Important note on `drift_triggered_job_run_id`: this value is a string (a job run identifier), not a number -- `mlflow.log_metrics()` only accepts numeric values and will raise an error if you try to log a string through it. Log this value with `mlflow.log_param()` instead, as shown below.

In [11]:

```
latencies_ms = []
cache_miss_count = 0
psi_scores_observed = []
drift_triggered_job_run_id = None
rolling_window = []

start_time = time.time()
with mlflow.start_run(run_name="stress_test_run") as run:
    for i in range(TOTAL_REQUESTS):
        item_id = sample_items.iloc[i % N_ITEMS]["item_id"]
        in_shift_window = SHIFT_START <= i < SHIFT_START + SHIFT_ROWS

        t0 = time.perf_counter()
        values, was_miss = hydrate_feature_vector(item_id)
        if in_shift_window:
            shift_idx = FEATURE_COLUMNS.index(SHIFTED_COLUMN)
            values[shift_idx] = float(np.random.beta(6, 1.5))
        latency_ms = (time.perf_counter() - t0) * 1000
        latencies_ms.append(latency_ms)
        if was_miss:
            cache_miss_count += 1

        rolling_window.append(values[FEATURE_COLUMNS.index(SHIFTED_COLUMN)])
        if len(rolling_window) > PSI_WINDOW_SIZE:
            rolling_window.pop(0)

        if (i + 1) % PSI_CHECK_INTERVAL == 0 and len(rolling_window) >= PSI_WINDOW_SIZE:
            psi = calculate_psi(baseline_df[SHIFTED_COLUMN].values, np.array(rolling_window))
            psi_scores_observed.append(psi)
            if psi > PSI_THRESHOLD and drift_triggered_job_run_id is None:
                drift_triggered_job_run_id = trigger_retraining_job_stub()
                mlflow.log_metric("psi_at_trigger", psi, step=i)

    total_duration_sec = time.time() - start_time

    if drift_triggered_job_run_id is not None:
        reg_result = register_model_stub(run.info.run_id)
        mlflow.set_tag("model_registration_version", reg_result["version"])
        endpoint_result = update_endpoint_stub(reg_result["version"])
        mlflow.set_tag("endpoint_update_status", endpoint_result["status"])

    p95_latency_ms = float(np.percentile(latencies_ms, 95))
    cache_miss_rate = cache_miss_count / TOTAL_REQUESTS
    psi_max_score = max(psi_scores_observed) if psi_scores_observed else 0.0
    saves_rate, dwell_time_median_s, private_shares_rate = simulate_engagement_metrics(
        drift_active=(drift_triggered_job_run_id is not None)
    )

    telemetry_summary = {
        "p95_serving_latency_ms": p95_latency_ms,
        "cache_miss_rate": cache_miss_rate,
        "psi_max_score": psi_max_score,
        "saves_rate": saves_rate,
        "dwell_time_median_s": dwell_time_median_s,
        "private_shares_rate": private_shares_rate,
    }
    mlflow.log_metrics(telemetry_summary)
    if drift_triggered_job_run_id is not None:
        mlflow.log_param("drift_triggered_job_run_id", drift_triggered_job_run_id)

    print(f"Total requests processed: {TOTAL_REQUESTS}")
    print(f"Total wall-clock time: {total_duration_sec:.1f}s")
    for k, v in telemetry_summary.items():
        print(f"{k}: {v:.4f}")
    print(f"drift_triggered_job_run_id: {drift_triggered_job_run_id}")
    print(f"\np95 SLA ({P95_SLA_MS}ms) met: {p95_latency_ms < P95_SLA_MS}")
    print(f"Drift trigger fired during shift window: {drift_triggered_job_run_id is not None}")
    print(f"\nMLflow run ID: {run.info.run_id}")
```

```
Total requests processed: 30000
Total wall-clock time: 4.5s
p95_serving_latency_ms: 0.3071
cache_miss_rate: 0.0493
psi_max_score: 9.7172
saves_rate: 0.1001
dwell_time_median_s: 34.7556
private_shares_rate: 0.0181
drift_triggered_job_run_id: 6ef0507f-prod-retrain

p95 SLA (50ms) met: True
Drift trigger fired during shift window: True

MLflow run ID: 8fd5d7b918d74a00810e0b5d9683be95
```

Post-run verification and diagnostics

Reads the run back from MLflow to confirm the six required metrics, the `drift_triggered_job_run_id` parameter and the two CI/CD tags were recorded, then summarises the PSI trajectory and latency distribution from the stress test above. Nothing in this section logs anything further to the run.

In [12]:

```python
from mlflow.tracking import MlflowClient

client = MlflowClient()
logged = client.get_run(run.info.run_id)
REQUIRED_METRICS = ["p95_serving_latency_ms", "cache_miss_rate", "psi_max_score",
                    "saves_rate", "dwell_time_median_s", "private_shares_rate"]

print(f"Experiment: {client.get_experiment(logged.info.experiment_id).name}")
print(f"Run ID:     {logged.info.run_id}  (status {logged.info.status})")
print(f"All six required metrics logged: {all(m in logged.data.metrics for m in REQUIRED_METRICS)}")
for m in REQUIRED_METRICS:
    print(f"  {m}: {logged.data.metrics[m]:.4f}")
print(f"Parameter drift_triggered_job_run_id: {logged.data.params.get('drift_triggered_job_run_id')}")
print(f"Tag model_registration_version: {logged.data.tags.get('model_registration_version')}")
print(f"Tag endpoint_update_status:     {logged.data.tags.get('endpoint_update_status')}")
trigger_hist = client.get_metric_history(run.info.run_id, "psi_at_trigger")
for h in trigger_hist:
    print(f"Metric psi_at_trigger: {h.value:.4f} at request index {h.step}")
```

```
Experiment: arcturus-pipeline-validation
Run ID:     8fd5d7b918d74a00810e0b5d9683be95  (status FINISHED)
All six required metrics logged: True
  p95_serving_latency_ms: 0.3071
  cache_miss_rate: 0.0493
  psi_max_score: 9.7172
  saves_rate: 0.1001
  dwell_time_median_s: 34.7556
  private_shares_rate: 0.0181
Parameter drift_triggered_job_run_id: 6ef0507f-prod-retrain
Tag model_registration_version: 69e97ecf
Tag endpoint_update_status:     Updated
Metric psi_at_trigger: 2.2366 at request index 15499
```

In [13]:

```
# PSI trajectory: check k ends at request (PSI_WINDOW_SIZE + k * PSI_CHECK_INTERVAL)
check_end = np.array([PSI_WINDOW_SIZE + k * PSI_CHECK_INTERVAL for k in range(len(psi_scores_observed))])
psi_arr = np.array(psi_scores_observed)
win_start = check_end - PSI_WINDOW_SIZE
shift_end = SHIFT_START + SHIFT_ROWS
pre = check_end <= SHIFT_START
full = (win_start >= SHIFT_START) & (check_end <= shift_end)
post = win_start >= shift_end
over = psi_arr > PSI_THRESHOLD
first = int(np.argmax(over))

print(f"PSI checks run: {len(psi_arr)} (every {PSI_CHECK_INTERVAL} requests, {PSI_WINDOW_SIZE}-row window)
")
print(f"  Before shift  (window entirely pre-shift) : max {psi_arr[pre].max():.4f}, mean {psi_arr[pre].mea
n():.4f}")
print(f"  First breach  : PSI {psi_arr[first]:.4f} at request {check_end[first]:,} "
      f"({check_end[first] - SHIFT_START:,} requests after the shift began)")
print(f"  Fully shifted window                       : min {psi_arr[full].min():.4f}, max {psi_arr[full].m
ax():.4f}")
print(f"  After shift   (window entirely post-shift) : max {psi_arr[post].max():.4f}")
print(f"  Checks above threshold: {int(over.sum())}; retraining jobs triggered: "
      f"{1 if drift_triggered_job_run_id else 0}")

lat = np.array(latencies_ms)
print(f"\nServing-path latency over {len(lat):,} requests (ms): "
      f"p50 {np.percentile(lat, 50):.4f} | p95 {np.percentile(lat, 95):.4f} | "
      f"p99 {np.percentile(lat, 99):.4f} | max {lat.max():.4f}")
print(f"p95 headroom vs {P95_SLA_MS} ms SLA: {P95_SLA_MS / np.percentile(lat, 95):,.0f}x")
print(f"Effective throughput: {TOTAL_REQUESTS / total_duration_sec:,.0f} requests/s "
      f"({TOTAL_REQUESTS:,} requests in {total_duration_sec:.1f}s vs a {config['qps_target']} QPS target)"
)
```

```
PSI checks run: 59 (every 500 requests, 1000-row window)
  Before shift  (window entirely pre-shift) : max 0.0145, mean 0.0079
  First breach  : PSI 2.2366 at request 15,500 (500 requests after the shift began)
  Fully shifted window                       : min 8.6590, max 9.7172
  After shift   (window entirely post-shift) : max 0.0145
  Checks above threshold: 21; retraining jobs triggered: 1

Serving-path latency over 30,000 requests (ms): p50 0.0609 | p95 0.3071 | p99 0.5060 | max 1.
7179
p95 headroom vs 50 ms SLA: 163x
Effective throughput: 6,613 requests/s (30,000 requests in 4.5s vs a 100 QPS target)
```

Supplementary: hit-path vs. miss-path latency

The harness records one latency per request but not which path served it, so this cell replays 20,000 lookups through `hydrate_feature_vector` and splits the timings by path. It is a post-run micro-benchmark, not part of the logged MLflow run.

In [14]:

```
item_ids = sample_items["item_id"].tolist()  # looked up outside the timed region
hit_ms, miss_ms = [], []
for j in range(20000):
    item_id = item_ids[j % N_ITEMS]
    t0 = time.perf_counter()
    _, was_miss = hydrate_feature_vector(item_id)
    (miss_ms if was_miss else hit_ms).append((time.perf_counter() - t0) * 1000)

for label, arr in [("Redis hit path ", hit_ms), ("Delta miss path", miss_ms)]:
    arr = np.array(arr)
    print(f"{label}: n={len(arr):>6,} | p50 {np.percentile(arr, 50):.4f} ms | p95 {np.percentile(arr, 95):
.4f} ms")
print(f"Miss path p50 is {np.median(miss_ms) / np.median(hit_ms):.1f}x the hit path p50 in this lab enviro
nment.")
```

```
Redis hit path : n=18,981 | p50 0.0533 ms | p95 0.1019 ms
Delta miss path: n= 1,019 | p50 0.4471 ms | p95 0.7642 ms
Miss path p50 is 8.4x the hit path p50 in this lab environment.
```

Step 5: Executive Summary

Contoso — Production Pipeline Architecture

Executive Summary

Author: Dennis Lam

Date: October 17, 2026

Pipeline Version: 1.0.0

1. Architecture Overview

Contoso's Arcturus recommendation pipeline had four components built in isolation and never tested together under load. Layer 1 hydrates features from Azure Cache for Redis, falling back to the Delta feature store on misses. Layer 2 registers retrained models with Azure ML SDK v2 and deploys them to a Managed Online Endpoint. Layer 3 compares a rolling window of served features with the Delta baseline using PSI, triggering the Databricks Jobs API past threshold. Layer 4 logs latency, cache, drift and engagement telemetry to MLflow, closing one automated loop from serving to redeployment.

2. Stress Test Results

The harness processed all 30,000 requests, the 100 QPS × 300-second volume, in 4.5 seconds on a single kernel. p95 serving latency was 0.3071 ms against the 50 ms SLA, with a cache miss rate of 0.0493. Before the shift, PSI never exceeded 0.0145. The first check inside the 10,000-row shift scored 2.2366 and triggered retraining job 6ef0507f-prod-retrain 500 requests after injection; PSI peaked at 9.7172, then fell back to 0.0145. Twenty-one checks breached 0.25, yet only one job fired. Both acceptance criteria were met.

3. Cost vs. Latency Trade-Off Defense

The Redis configuration I am defending is sized for memory, not throughput: hold enough of the serving corpus to keep misses at or below 5%. The run supports that. Cache hits had a p50 of 0.0533 ms against 0.4471 ms for the feature-store fallback, 8.4 times slower. With misses at 0.0493, p95 (0.3071 ms) sits at the boundary between the hit and miss distributions, so a cheaper, smaller cache that let misses pass 5% would push p95 onto the fallback path. Throughput gives no case for a premium tier: the path cleared 6,613 requests per second, 66 times the 100 QPS target, with p95 163 times under the SLA. I would choose the smallest replicated tier whose memory holds the hot set.

I kept the PSI threshold at 0.25 because the margins are wide on both sides: the noise floor peaked at 0.0145, while the first shifted check scored 2.2366. A lower threshold would not have caught this shift sooner, since detection already came within one 500-request interval, but it would raise false-retrain exposure, and every trigger costs a Databricks job plus an endpoint redeployment; the fire-once guard held 21 breaches to one job. On cost, the 50 ms target is not what drives spend: even the fallback's p95 of 0.7642 ms still passes it. That fallback is in-memory, however, so I would not drop the cache for the cheaper cache-less design until real Delta reads are measured.

4. Production Readiness Assessment

The pipeline meets both acceptance criteria as the harness defines them: p95 latency of 0.3071 ms against a 50 ms SLA, and an automatic retraining trigger 500 requests into the 10,000-row shift, so I recommend proceeding with the deployment window. The single highest residual risk is drift the loop cannot see. Only user_engagement_score is monitored, so a shift in the other five features would degrade recommendations without triggering retraining. In the first 30 days I would monitor PSI on all six columns at the same 0.25 threshold.

5. MLflow Experiment Run Reference

Experiment Name: arcturus-pipeline-validation

Run ID: 8fd5d7b918d74a00810e0b5d9683be95

Key Metrics Logged:

- p95_serving_latency_ms: 0.3071
- cache_miss_rate: 0.0493
- psi_max_score: 9.7172
- saves_rate: 0.1001
- dwell_time_median_s: 34.7556
- private_shares_rate: 0.0181

Key Parameter Logged:

- drift_triggered_job_run_id: 6ef0507f-prod-retrain